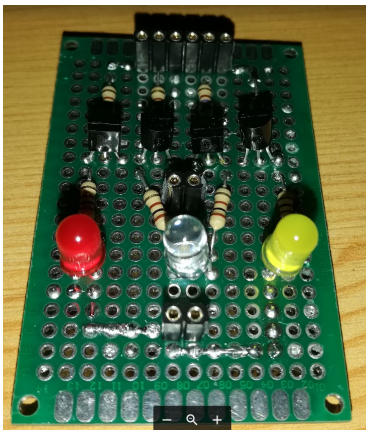
PLA9: Integración de tecnologías y plan de negocio

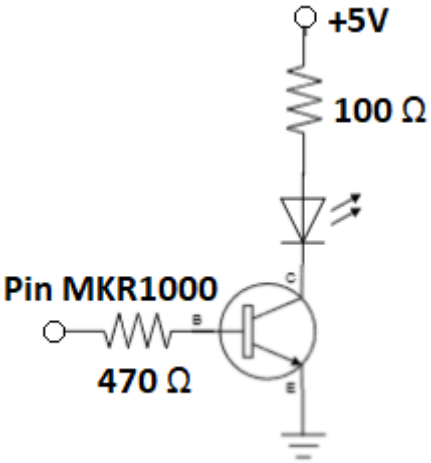
Hardware
MKR1000

En la parte local de la solución hemos utilizado la MKR1000 para el control de los actuadores (en este caso la resistencia que calienta el agua y el motor que la bombea) y del sensor que mide la temperatura del agua. Tanto la resistencia como el motor son accionados por dos relés que permiten controlar cargas que la MKR1000 no puede controlar por si misma.

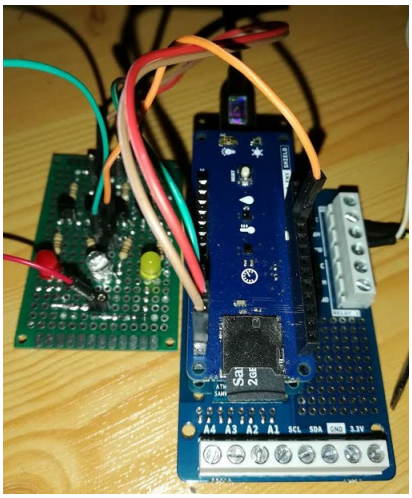
Adicionalmente hemos conectado un pequeño panel desarrollado a medida que permite visualizar el estado del sistema.



Este panel consta de 3 LEDs (uno de ellos bicolor). Los leds son controlados por cuatro salidas digitales de la MKR1000. Para evitar superar la corriente máxima que puede proporcionar cada una de estas salidas (aprox. 7mA), el circuito incluye cuatro transistores (con dos resistencias cada uno para su correcta polarización en saturación) que proporcionan la corriente necesaria a los LEDs (aprox. 25mA cada uno).



En esta segunda imagen se muestra el panel conectado a la MKR1000. El panel se alimenta con una fuente de 5V independiente.

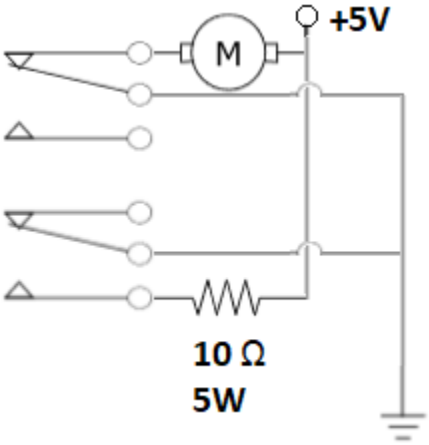


Los LEDs permiten conocer el estado del sistema:

- Un LED amarillo que se enciende cuando se reciben comandos mediante MQTT para, por ejemplo, actualizar el estado de los relés o el valor del termostato.
- Un LED rojo que se enciende cuando la resistencia responsable de calentar el agua está activada.
- Un LED rojo / verde (bicolor) que indica el estado del sistema. Rojo cuando encendemos el equipo y verde cuando tiene conectividad WiFi y ha establecido conexión con el servidor MQTT.

Para este ejercicio se han utilizado un pequeño motor eléctrico que simulará la bomba de agua y una resistencia que proporcionará calor suficiente para aumentar la temperatura del sensor. Tanto el motor como la resistencia se alimentan con la fuente independiente que también alimenta al panel de control.
Esta fuente proporciona los 200mA necesarios para mover el motor y aprox. 500mA a la resistencia. Con estos valores, la resistencia disipará aproximadamente 2.5W en forma de calor. Por esta razón se ha seleccionado un modelo de 5W que soportará el calor generado.

Como curiosidad, las resistencias de los transistores son tan solo de 0.25W (se estima que la de 100Ω disipará unos 0.06W).



Raspberry Pi

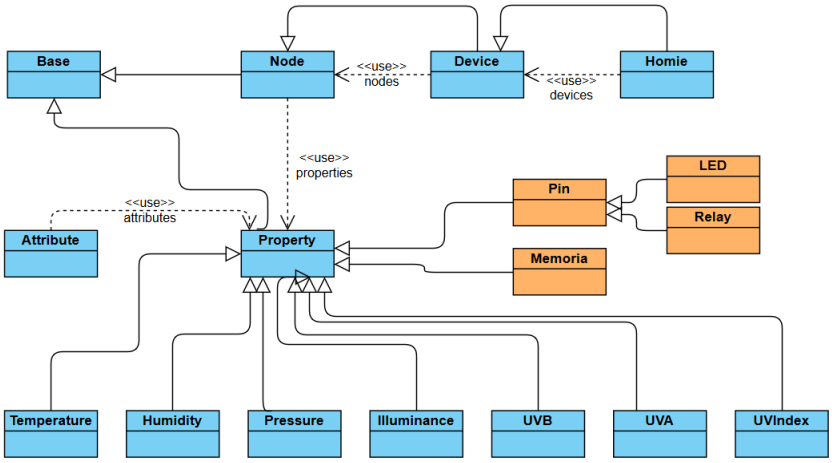
Se utiliza en esta solución una Raspberry Pi que hace funciones de gateway: proporciona conectividad WiFi a la MKR1000, un Mosquitto que recibe los mensajes MQTT, almacena estos mensajes en una base de datos InfluxDB y permite su monitorización utilizando Grafana.
La instalación y configuración de estos productos ya se describió en un ejercicio anterior.

Software

MKR1000

El código instalado en la MKR1000 recoge la información de los sensores y la envía al broker MQTT. También escucha los mensajes MQTT que controlan el valor del termostato y que permiten cambiar de forma manual el estado de los relés. Todo el código se encuentra en **http://github.com/gnietoUOC/PLA9**.

Para la generación y recepción de los mensajes MQTT se utiliza una librería propia que permite gestionarlos de forma automática. Al iniciar la MKR1000, el código modela los sensores actuadores disponibles.



Ahora, además de monitorizar ciertas variables, en cada ejecución del bucle principal se comprueba si la temperatura es inferior a la seleccionada en el termostato. En ese caso, se activa el calentador. Cuando la temperatura supera la del termostato (más un cierto margen

que actualmente es de 0.5º) el calentador se apaga. Junto con el calentador se activa o desactiva el LED de color rojo que hay en el panel de control.

```
void loop() {
  if (millis()>next1) {
    next1 = millis()+PERIOD1;
    homie->update();
  }

  if (millis()>next2) {
    next2 = millis()+PERIOD2;

   // Desactivo el calentador si se ha superado la temperatura
   if (t->getValue() > th->getIValue()+OFFSET) {
      r1->set(false);
      rl->set(false);
    }

    // Activo el calentador si la temperatura está por debajo
    // de la del termostato
    if (t->getValue() < th->getIValue()) {
      r1->set(true);
      rl->set(true);
    }
  }

  homie->reconnect();

  client->loop();

}
```

Cache

Cada 5sg se comprueba el valor de los sensores (temperatura, humedad, …) y se envían a MQTT. Si en el momento de publicar el mensaje con los datos se hubiera perdido la conexión (MQTT caído, Raspberry sin conectividad …) los datos se almacenan.

```
void Base::pub(char *tag,char *value) {
  char data[96];
  char path[96];

  DPRINTLN("-> Base.pub");
  // Si hay conexión, enviamos el dato
  if (getClient()->connected()) {
    getPath(path);
    if (tag!=NULL) {
      sprintf(data,"%s/%s",path,tag);
      if (value!=NULL) {
        client->publish(data,value,MQRETAIN);
      } else {
        client->publish(data,NULL,0,MQRETAIN);
      }
      DPRINTLN(data);
    } else {
      client->publish(path,value,MQRETAIN);
    }
  // Si no hay conexión almancenamos el dato
  } else {
    DPRINTLN("Recording...");
    Record *r = new Record(WiFi.getTime(),this,tag,value);
    getCache()->push(r);
  }
  DPRINTLN("<- Base.pub");
}
```

En el momento en el que se detecta que se ha restablecido la conexión, todos estos datos se envían a MQTT.

```
if(!getClient()->connected()) {
  Serial.print("Connecting MQTT...");
  // He modificado la conexión para definir un mensaje 'Last Will'
  if (getClient()->connect(MQCLIENT,MQUSERNAME,MQPWD,WILLTOPIC,1,false,WILLMESSAGE,CLEANSESSION)) {
    Serial.println("Connected");

    // Verificamos si la caché tiene registros pendientes de enviar
    Cache *cache = getCache();
    while (cache->length()>0) {
      Record *r = cache->peek();
      r->getBase()->pub(r);
    }
  } else {
```

La publicación de mensajes es ligeramente diferente porque ahora tenemos que extraer la información de la cache.

```
void Base::pub(Record *r) {
  char value[32];
  char path[96];

  DPRINTLN("-> Base.pub2");

  if (getClient()->connected()) {
    getPath(path);
// Supongo que las propiedades no se han perdido (overflow)
    sprintf(value,"%s@%ld",r->getValue(),r->getTime());
    Serial.println(value);
    client->publish(path,value,MQRETAIN);
    // Supongo que ha ido bien ... elimino el mensaje de la cache
    // y libero la memoria
    getCache()->pull();
    free(r);

    // Después de publicar el mensaje guardamos una entrada
    // en un fichero en la tarjeta
    logger(value);
  }

  DPRINTLN("<- Base.pub2");
}
```

Tan importante como el dato lo es el instante en el tiempo en que se está obteniendo. Es por esta razón que, en caso de perderse la conectividad, para cada muestra se guardan tres datos:

- El instante en el que se ha tomado la muestra.
- El elemento que la ha generado (sensor de temperatura, presión, …). Este dato es importante para poder reconstruir posteriormente el topic.
- El valor de la muestra.

```
class Record {

  public:
    Record(unsigned long time, Base *base, char *type, char* value);

   long getTime();
   Base *getBase();
   char *getTag();
   char *getValue();

  private:
    unsigned long time;
    char value[8];
    char tag[16];
    Base *base;


};
```

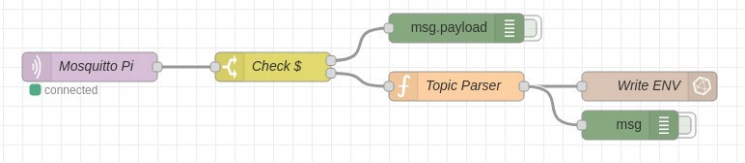
A la hora de enviar la información a MQTT, la convención Homie no contempla la posibilidad de enviar estructuras de datos ni timestamps. Y no podemos enviar más de un mensaje porque necesitamos que la información llegue ‘a la vez’: valor e instante.
Para resolver este problema se concatena en el mensaje el valor medido junto con el timestamp: <valor>@<timestamp>. Para evitar problemas relacionados con la utilización de

diferentes husos horarios (y facilitar la extracción de la información), el timestamp se envía en formato numérico (milisegundos) y no texto (HH:mm:ss.mmm DD/MM/YYYY).

Raspberry Pi

En la Raspberry Pi hemos desarrollado dos flujos con Node-RED. Estos flujos se encuentran en **http://github.com/gnietoUOC/PLA9-Node**.
El primero de estos flujos se encarga de recoger los datos que llegan a Mosquitto y enviarlos a la base de datos InfluxDB. El segundo flujo es el responsable de crear el Dashboard.

Mosquitto MQTT -> InfluxDB



El nodo **Check$** se encarga de separar los mensajes MQTT que describen propiedades de los que tienen datos.
El nodo **Topic Parser** realizar dos operaciones. La primera es separar los diferentes campos Homie del topic para utilizarlos como etiquetas a la hora de guardar los registros en InfluxDB. La segunda es verificar si el dato es un dato 'simple' o incluye el timestamp (mensajes que por alguna incidencia han llegado con retraso). En este segundo caso, se incluye el timestamp en el 'payload' de Node-RED para que InfluxDB lo utilice (y no el instante en el que llega el mensaje).

```
var a = msg.topic.indexOf('/',6);
var b = msg.topic.indexOf('/',a+1);

var c = msg.payload.indexOf('@');

// Debug data
msg.a = a;
msg.b = b;
msg,c = c;

var device = msg.topic.substring(6,a);
var node = msg.topic.substring(a+1,b);
var token = msg.topic.substring(b+1);
var value = parseFloat(c>0? msg.payload.substring(0,c):msg.payload);
// La MKR1000 da el timestamp en segundos, Node-RED en milisegundos.
// InfluxDB espera nanosegundos así que tenemos que hacer la conversión.
var timestamp = c>0? parseInt(msg.payload.substring(c+1))*1e9:new Date().getTime()*1e6;

msg.payload = [{
    value: value,
    time: timestamp
},
{
    tag1:device,
    tag2:node,
    tag3:token
}];

return msg;
```

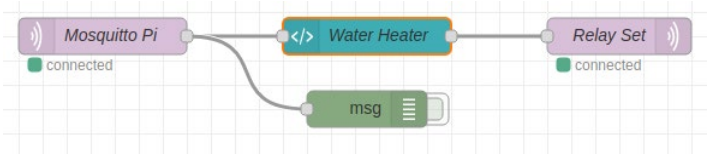
Los datos que estamos guardando en InfluxDB es necesario, según los requerimientos de este ejercicio, conservarlos sólo durante 24 horas. Para ello lo que se ha hecho es definir una 'retention policy' en InfluxDB con el siguiente comando:

```
CREATE RETENTION POLICY "24H" ON "ENV" DURATION 24h REPLICATION 1 DEFAULT
```

```
Connected to https://localhost:8086 version 1.7.9
InfluxDB shell version: 1.7.9
> CREATE RETENTION POLICY "24H" ON "ENV" DURATION 24h REPLICATION 1 DEFAULT
>
```

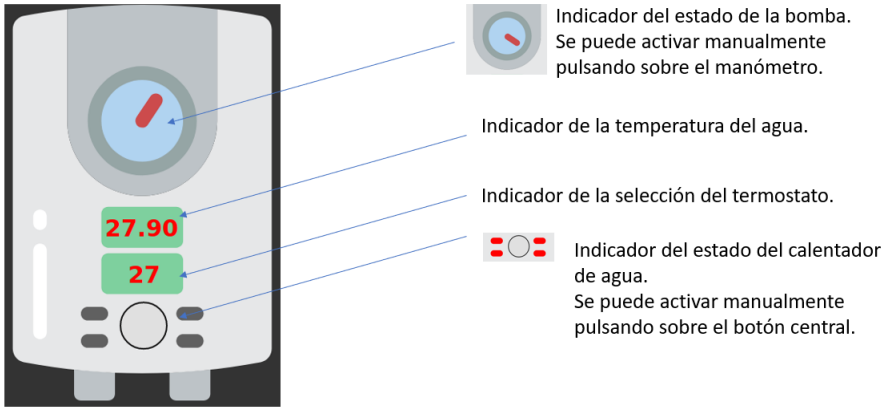
Dashboard

También, se ha creado un pequeño flujo para proporcionar un dashboard que muestre el estado de funcionamiento del sistema.



Este dashboard únicamente muestra un dibujo del calentador del agua.

En él se puede ver tanto información del estado del motor como del calentador del agua. No sólo podemos monitorizar las temperaturas, la del agua y la seleccionada en el termostato, sino que también podemos interactuar con el panel para activar la bomba o el calentador de forma manual.

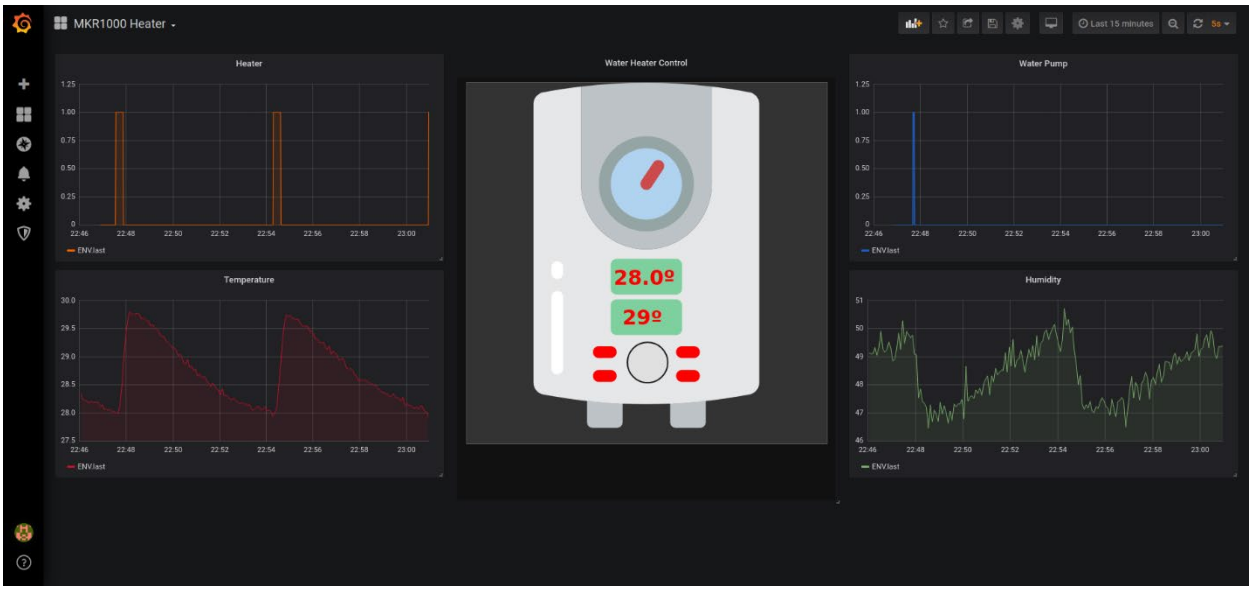


Para conseguir un panel con el que podemos interactuar se ha utilizado un fichero SVG[1] al que se ha añadido JavaScript para procesar los mensajes MQTT, capturar las pulsaciones sobre la imagen y actualizar el aspecto.
Este primer fragmento de código captura los mensajes que llegan al nodo del dashboard. En función del topic actualiza el dibujo.

```
myScope.$watch("msg",function(msg) {
    if (msg) {
        if (msg.topic.includes('Temperature')) {
            temp.text(formatTemp(msg.payload));
        }

        if (msg.topic.includes('Thermostat')) {
            thermo.text(formatTemp(msg.payload));
        }

        if (msg.topic.includes('Relay01')) {
            console.log("Relay01");
            lights.status = msg.payload=="1"? "ON":"OFF";
            var pilot = $("#lights");
            pilot[0].attributes["fill"].nodeValue =
                lights.status=="OFF"? "#606060":"#ff0000";
        }

        if (msg.topic.includes('Relay02')) {
            console.log("Relay02");
            manual.status = msg.payload=="1"? "ON":"OFF";
            var pilot = $("#pointer");
            pilot[0].attributes["transform"].nodeValue=
                "rotate("+(manual.status=="OFF"? 0:90)+",168,152)";
        }
    }
});
```

[1] Se ha modificado el SVG gratuito de la página https://www.flaticon.com/free-icon/water-heater_1677140 para ajustarlo a las necesidades de este ejercicio.

En el segundo fragmento es donde se capturan las pulsaciones sobre el dibujo para activar los relés de forma manual.

```
pump.click(function() {
    pump.status = pump.status=="OFF"? "ON":"OFF";
    myScope.send({
        payload:pump.status,
        topic:"Homie/MKR1000/MKRRELAY/Relay02/Set"
    });
});

manual.click(function() {
    manual.status = manual.status=="OFF"? "ON":"OFF";
    myScope.send({
        payload:manual.status,
        topic:"Homie/MKR1000/MKRRELAY/Relay01/Set"
    });
});
```

Finalmente, este dashboard de Node-RED se ha integrado dentro de un panel de Grafana en el que podemos ver el estado de los dos relés y la evolución de temperatura / humedad. Este panel es el que permite al técnico verificar el correcto funcionamiento de la instalación.